

Time Complexity Notes

1. Layman Explanation 🌟📊🔍

Imagine you're doing simple everyday tasks and think about how they get harder when you have more stuff:

O(1): Constant Time

Grabbing your toothbrush from the same cup every morning—always the same quick action, whether there's one toothbrush or a hundred.

O(log n): Logarithmic Time

Looking for a name in a big phone book by jumping to the middle, then to the middle of the remaining section, and so on—each jump halves the pages you look at.

O(n): Linear Time

Checking every book on a shelf one by one to find your favorite—you look at each book exactly once.

O(n log n): Linearithmic Time

Organizing a deck of cards by repeatedly splitting the deck in half and then merging sorted halves—faster than pure one-by-one sorting.

O(n²): Quadratic Time

Comparing every student in a class with every other student to see who shares the same birthday—you check each pair.

O(n² log n): Quadratic-Logarithmic Time

For each student (n² pairs), you then look up something quickly in a sorted list (binary search)—combines pairwise checks with halving searches.

O(n³): Cubic Time

Planning seating charts by considering every trio of guests—checking all possible triplets.

O(n³ log n): Cubic-Logarithmic Time

For every trio, you also do a quick lookup in a sorted list—triple loops plus halving steps inside.

O(n⁴): Quartic Time

Checking every group of four people at a party—considering all possible quadruplets.

2. Ranking (Fastest → Slowest) ⚡📊🔴

(Fastest → Slowest)

1. O(1) (constant)
2. O(log n) (logarithmic)
3. O(n) (linear)
4. O(n log n) (linearithmic)
5. O(n²) (quadratic)
6. O(n² log n)
7. O(n³) (cubic)
8. O(n³ log n)
9. O(n⁴) (quartic)

Generally, anything beyond O(n²) is rare in practical code and often too slow for large n.

3. Indicators (Code "Fingerprints") 🧠🧪📖

O(1): No loops/recursion; single lookups or arithmetic.

O(log n): Problem size halves (or constant factor reduction) each step; binary search or tree ops.

O(n): Single loop/recursion visiting each element once.

O(n log n): Divide & conquer with O(n) work each level; merge sort, heap operations.

O(n²): Two nested loops over n; pairwise comparisons.

O(k loops): k nested loops → O(nᵏ).

Loop + recursive/log work: Combine by multiplication: e.g., O(n) × O(log n) = O(n log n).

4. How to Combine Complexities 🔗+✖

Understanding how to combine multiple parts of a program is crucial for accurate time complexity estimation.

🔗 When Do You Multiply?

You multiply complexities when:

One operation is nested inside another.

💡 **Example:**

```
for x in arr:           # O(n)
    binary_search(arr, x) # O(log n)
```

Total complexity: O(n × log n) → O(n log n)

+ When Do You Add?

You add when:

You have separate, non-nested operations.

💡 **Example:**

```
def func(arr):
    for x in arr:      # O(n)
        print(x)
    for y in arr:      # O(n)
        print(y)
```

Total complexity: O(n + n) → O(2n) → Drop constant → O(n)

🎯 Rule of Thumb:

Add → when steps are separate
Multiply → when steps are nested
Always keep the biggest term (dominant)

🚫 Why No Addition Table?

While it's useful to understand additive behavior conceptually, we don't need a formal addition table in practice. That's because:

Big-O notation drops constants.

We always keep only the dominant term (largest one).

Addition just simplifies to the slowest-growing part.

So instead, we focus on the multiplication table – where actual structural growth occurs due to nesting – and remember the addition rule as a simple principle.

5. Time Complexity Multiplication Table 📊📌📐

×	O(1)	O(log n)	O(n)	O(n log n)	O(n²)
O(1)	O(1)	O(log n)	O(n)	O(n log n)	O(n²)
O(log n)	O(log n)	O((log n)²)	O(n log n)	O(n (log n)²)	O(n² log n)
O(n)	O(n)	O(n log n)	O(n²)	O(n² log n)	O(n³)
O(n log n)	O(n log n)	O(n (log n)²)	O(n² log n)	O(n² (log n)²)	O(n³ log n)
O(n²)	O(n²)	O(n² log n)	O(n³)	O(n³ log n)	O(n⁴)

6. Code Examples 🖥️🔧🔑

Below are sample functions illustrating each unique complexity from the table.

O(1) – Constant Time

```
# Always one step, no matter how big n is
def get_first(arr):
    return arr[0]
```

O(log n) – Logarithmic Time

```
# Repeatedly splits search range in half
def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid+1
        else:
            high = mid-1
    return -1
```

O(n) – Linear Time

```
# One loop through all items
def sum_list(arr):
    total = 0
    for x in arr:
        total += x
    return total
```

O(n log n) – Linearithmic Time

```
# Divide & conquer with O(n) merge at each of log n levels
def merge_sort(arr):
    if len(arr) < 2:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

O(n²) – Quadratic Time

```
# Two nested loops over n
def all_pairs(arr):
    result = []
    for x in arr:
        for y in arr:
            result.append((x, y))
    return result
```

O(n² log n) – Quadratic × Logarithmic

```
# Double loop, then binary search inside
from bisect import bisect_left

def nested_search(arr):
    for x in arr:
        for y in arr:
            idx = bisect_left(arr, y)
```

O(n³) – Cubic Time

```
# Three nested loops
def all_triplets(arr):
    triples = []
    for x in arr:
        for y in arr:
            for z in arr:
                triples.append((x, y, z))
    return triples
```

O(n³ log n) – Cubic × Logarithmic

```
# Triple loop with binary search inside
from bisect import bisect_left

def cube_search(arr):
    for x in arr:
        for y in arr:
            for z in arr:
                _ = bisect_left(arr, z)
```

O(n⁴) – Quartic Time

```
# Four nested loops
def all_quads(arr):
    quads = []
    for w in arr:
        for x in arr:
            for y in arr:
                for z in arr:
                    quads.append((w, x, y, z))
    return quads
```

7. TL;DR for Each Type 📌🔴🔒

O(1): Always constant, one step.

O(log n): Cuts problem in half each step.

O(n): One pass through all items.

O(n log n): Split & merge strategy (sorts).

O(n²): Nested loops → compare every pair.

O(n² log n): Pairs plus a logarithmic operation.

O(n³): Three-way nested loops (triplets).

O(n³ log n): Triplets with binary search.

O(n⁴): Four nested loops (quadruplets).